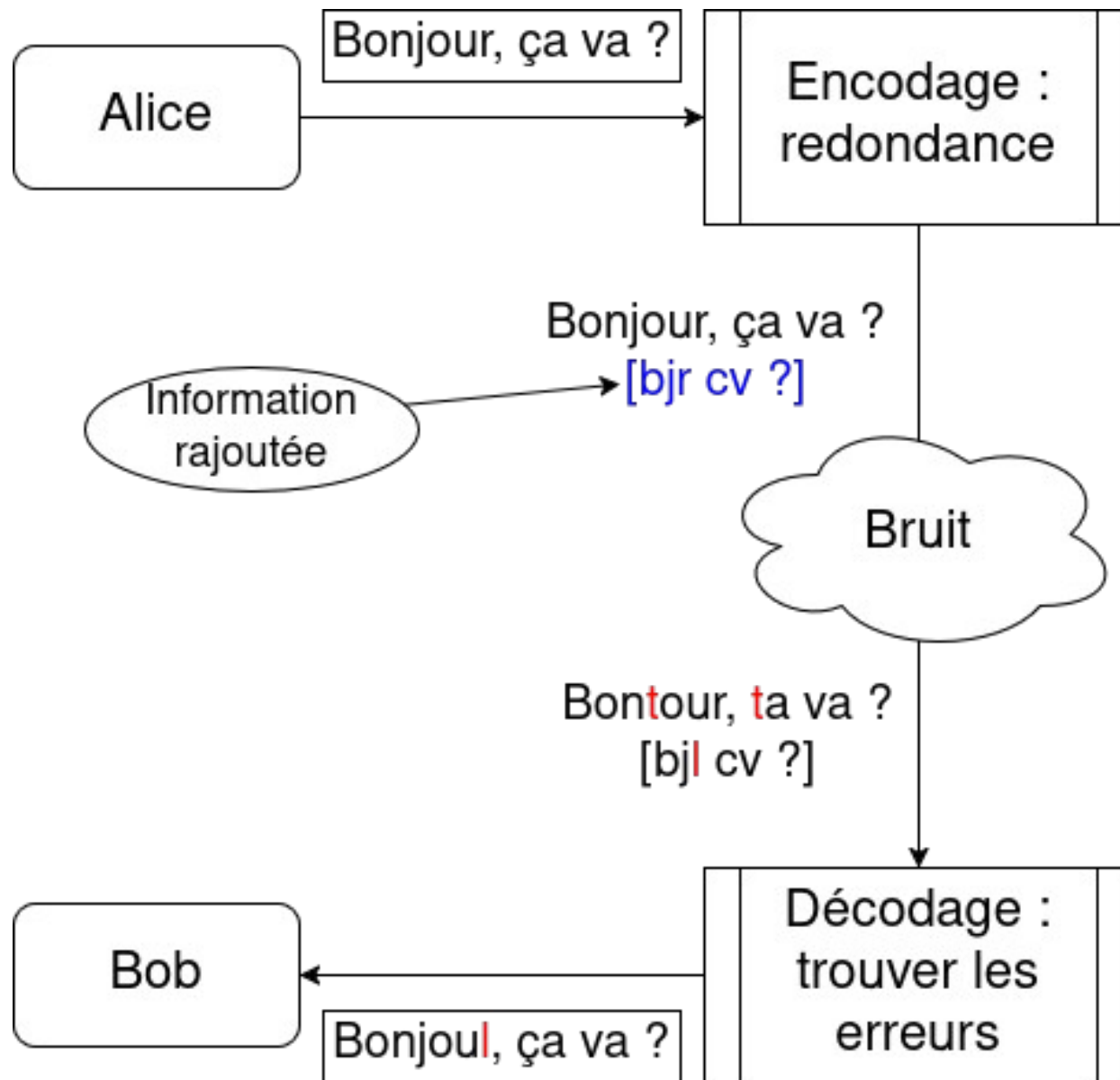


# Codes cycliques et Reed-Solomon

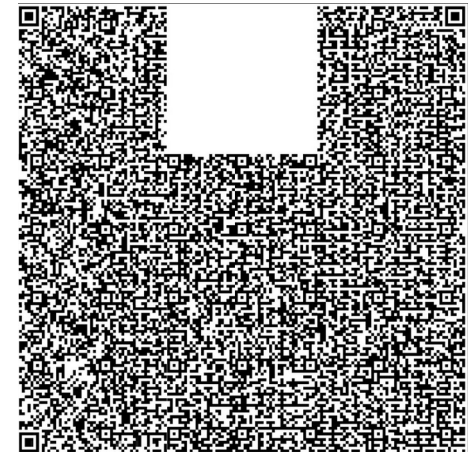
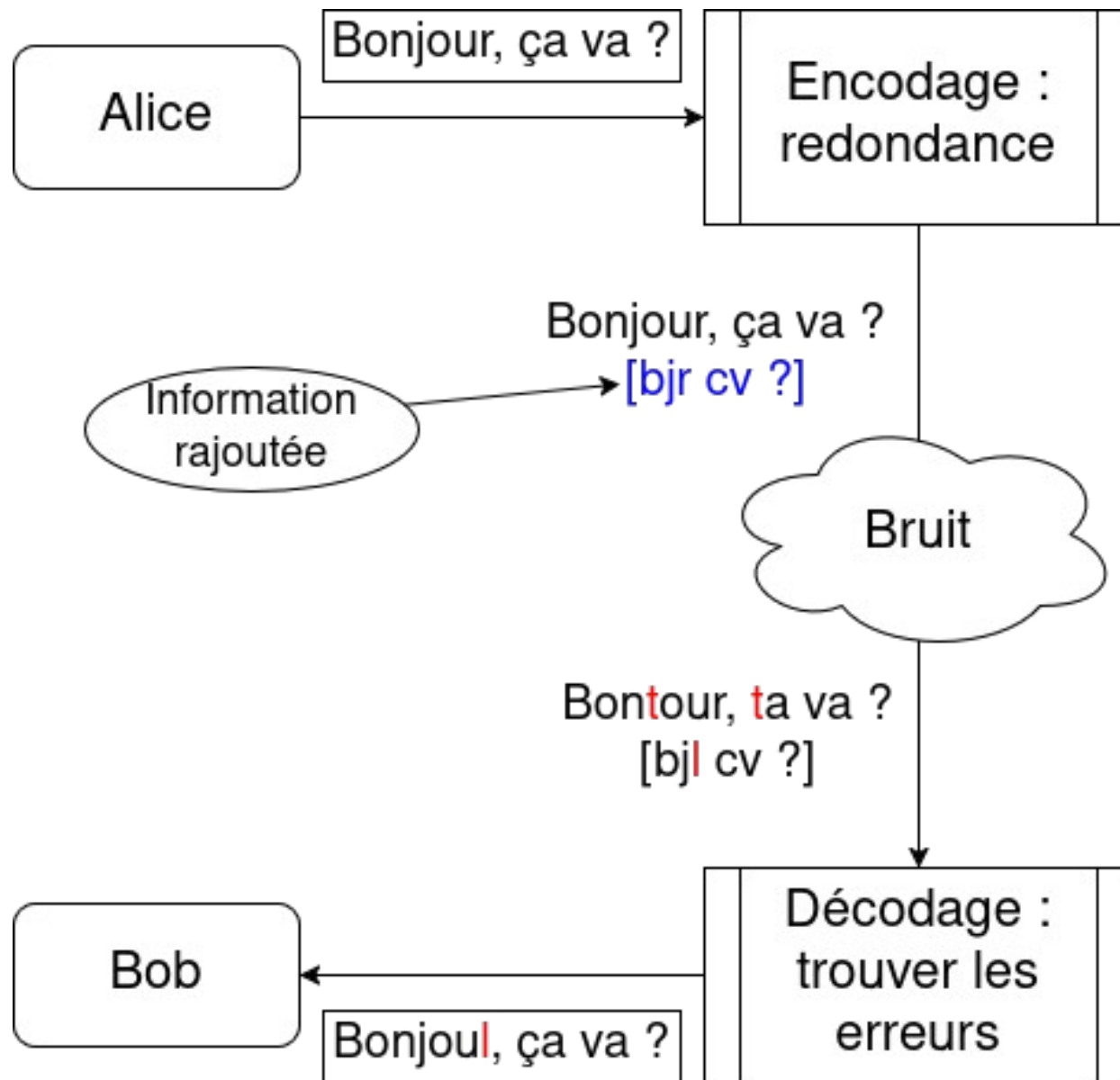
Philéas Lesourd 13720

en groupe avec :  
Vincent Rubin 20417

# Introduction



# Introduction

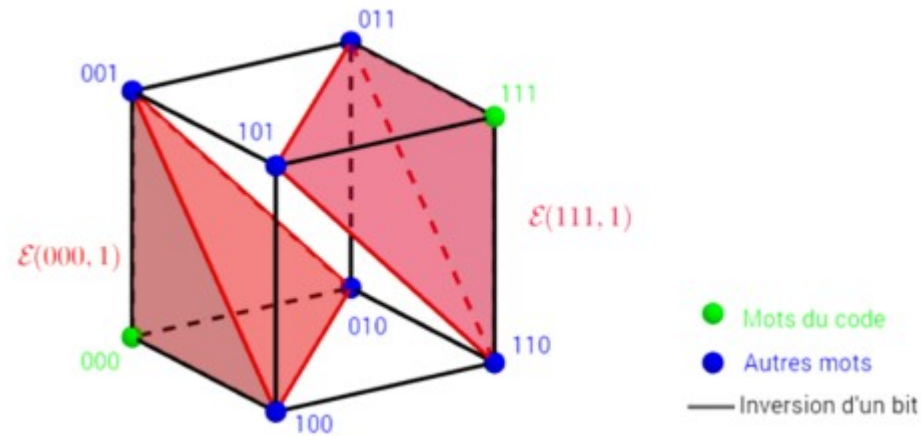


# Comment garantir l'intégrité d'un message et le corriger ?

- 1) Le code cyclique original de Reed-Solomon
- 2) Gorenstein-Zierler
  - a. Encodage
  - b. Polynôme d'erreur
  - c. Décodage
- 3) Résultats
- 4) Complexités et paramètres

# 1) Le code cyclique original de Reed-Solomon

- Code correcteur :
  - technique qui sert à assurer une *transmission* par un canal qui pourrait la *détériorer*.



# 1) Le code cyclique original de Reed-Solomon

- **Code linéaire :**
  - *espaces vectoriels* de dimension finie sur des *corps finis*.
  - applications injectives d'un espace vectoriel dans un autre de dimension *plus grande*.

# 1) Le code cyclique original de Reed-Solomon

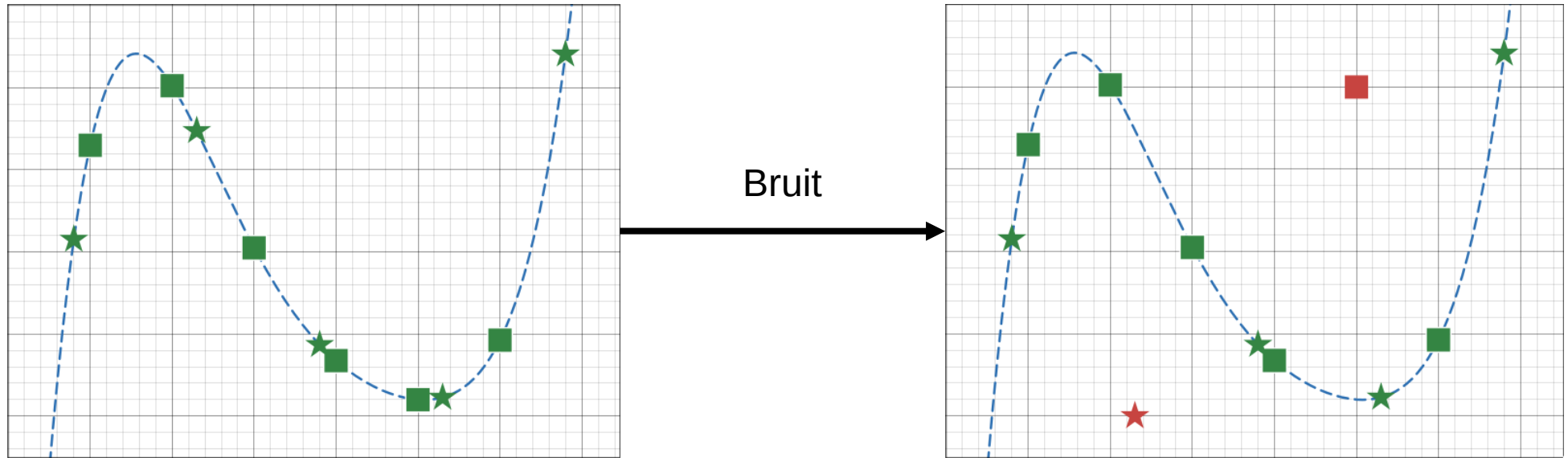
- **Code linéaire :**
  - *espaces vectoriels* de dimension finie sur des *corps finis*.
  - applications injectives d'un espace vectoriel dans un autre de dimension *plus grande*.
- **Code cyclique :**
  - **code linéaire**
  - **Si  $(c_1, \dots, c_n)$  est un mot encodé,  $(c_2, \dots, c_n, c_1)$  en est un.**

# 1) Le code cyclique original de Reed-Solomon

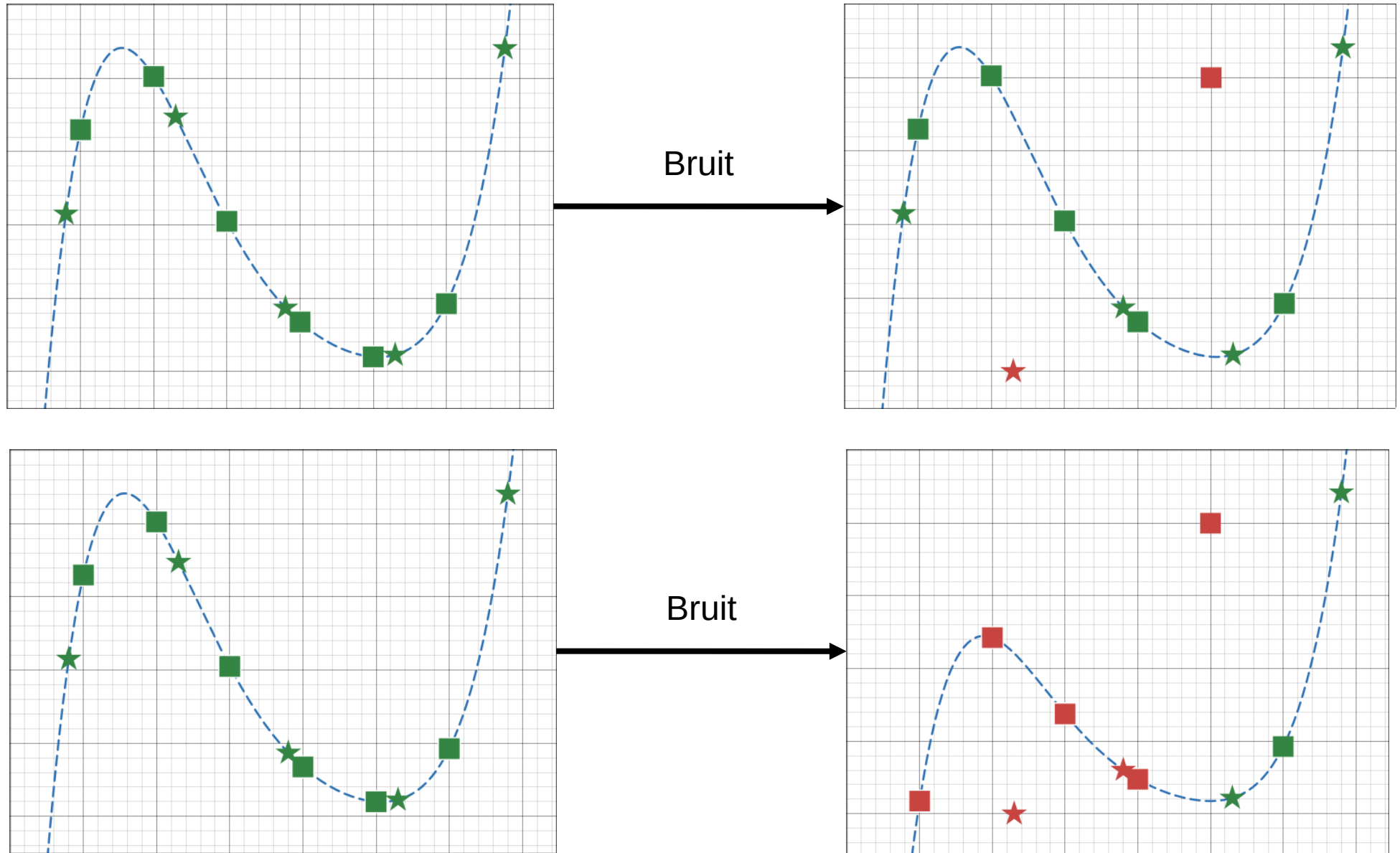
- On utilise  $\mathbb{F}_{256} = \mathbb{F}_2[X]/(Q)$  avec  $Q$  irréductible sur  $\mathbb{F}_2[X]$
- Les « mots » sont alors dans  $\mathbb{F}_{256}[X]$
- $\mathbb{F}_{256}^*$  est un groupe cyclique. On note  $\alpha = \overline{X}$  qui engendre  $\mathbb{F}_{256}^*$



# 1) Le code cyclique original de Reed-Solomon



# 1) Le code cyclique original de Reed-Solomon



## 2)a. Gorenstein-Zierler : encodage

Message de longueur  $k$  =  $A \in \mathbb{F}_{256}[X]$

$$G(X) = \prod_{j=1}^{2t} (X - \alpha^j) \quad \text{avec } 2t + k \leq 255$$

$$AX^{2t} = GQ + B$$

On encode  $A$  par :  $C = \overbrace{AX^{2t}}^{\text{k coeffs}} - \overbrace{B}^{\text{info rajoutée}} = GQ.$

$$\forall i \in \llbracket 1, 2t \rrbracket, C(\alpha^i) = 0$$

## 2)b. Gorenstein-Zierler : polynôme d'erreur

Polynôme d'erreur :  $E = D - C = \sum_{r=1}^{n_e} y_r X^{i_r}$

 connu       inconnu

On a  $n_e \leq t$  erreurs aux positions  $i_r$ .

$$\forall j \in [1, 2t], \quad S_j = D(\alpha^j) = E(\alpha^j) = \sum_{r=1}^{n_e} y_r (\alpha^j)^{i_r}$$

 connus       inconnus       inconnus

## 2)c. Gorenstein-Zierler : décodage

**1** On calcule les  $i_r$  :

$$S_j = \sum_{r=1}^{n_e} y_r (\alpha^j)^{i_r}$$

$$\Lambda(X) = \prod_{r=1}^{n_e} (1 - \alpha^{i_r} X) = 1 + \lambda_1 X + \lambda_2 X^2 + \dots + \lambda_{n_e} X^{n_e}$$

## 2)c. Gorenstein-Zierler : décodage

**1** On calcule les  $i_r$  :

$$S_j = \sum_{r=1}^{n_e} y_r (\alpha^j)^{i_r}$$

$$\Lambda(X) = \prod_{r=1}^{n_e} (1 - \alpha^{i_r} X) = 1 + \lambda_1 X + \lambda_2 X^2 + \dots + \lambda_{n_e} X^{n_e}$$

$$\begin{aligned} \sum_{i=1}^{\boxed{\nu}} S_{\nu+j-i} \lambda_i &= \sum_{i=1}^{\nu} \left( \sum_{r=1}^{\nu} y_r (\alpha^{i_r})^{\nu+j-i} \right) \lambda_i = \sum_{r=1}^{\nu} y_r (\alpha^{i_r})^{\nu+j} \sum_{i=1}^{\nu} (\alpha^{i_r})^{-i} \lambda_i \\ &= \sum_{r=1}^{\nu} y_r (\alpha^{i_r})^{\nu+j} (\Lambda(\alpha^{-i_r}) - 1) = - \sum_{r=1}^{\nu} y_r (\alpha^{i_r})^{\nu+j} \\ &= -S_{\nu+j} \end{aligned}$$

## 2)c. Gorenstein-Zierler : décodage

**1** On calcule les  $i_r$  :

$$S_j = \sum_{r=1}^{n_e} y_r (\alpha^j)^{i_r}$$

$$\sum_{i=1}^{\nu} S_{\nu+j-i} \lambda_i = -S_{\nu+j}$$

Pour  $\nu$  allant de  $t$  à 1 (dans l'ordre décroissant) :

$$\begin{pmatrix} S_{\nu} & S_{\nu-1} & \dots & S_1 \\ S_{\nu+1} & S_{\nu} & \dots & S_2 \\ \vdots & \vdots & \ddots & \vdots \\ S_{2\nu-1} & S_{2\nu-2} & \dots & S_{\nu} \end{pmatrix} \begin{pmatrix} \lambda_1 \\ \lambda_2 \\ \vdots \\ \lambda_{\nu} \end{pmatrix} = - \begin{pmatrix} S_{\nu+1} \\ S_{\nu+2} \\ \vdots \\ S_{2\nu} \end{pmatrix}$$

## 2)c. Gorenstein-Zierler : décodage

**2** On calcule les  $y_r$  :

$$S_j = \sum_{r=1}^{n_e} y_r (\alpha^j)^{i_r}$$

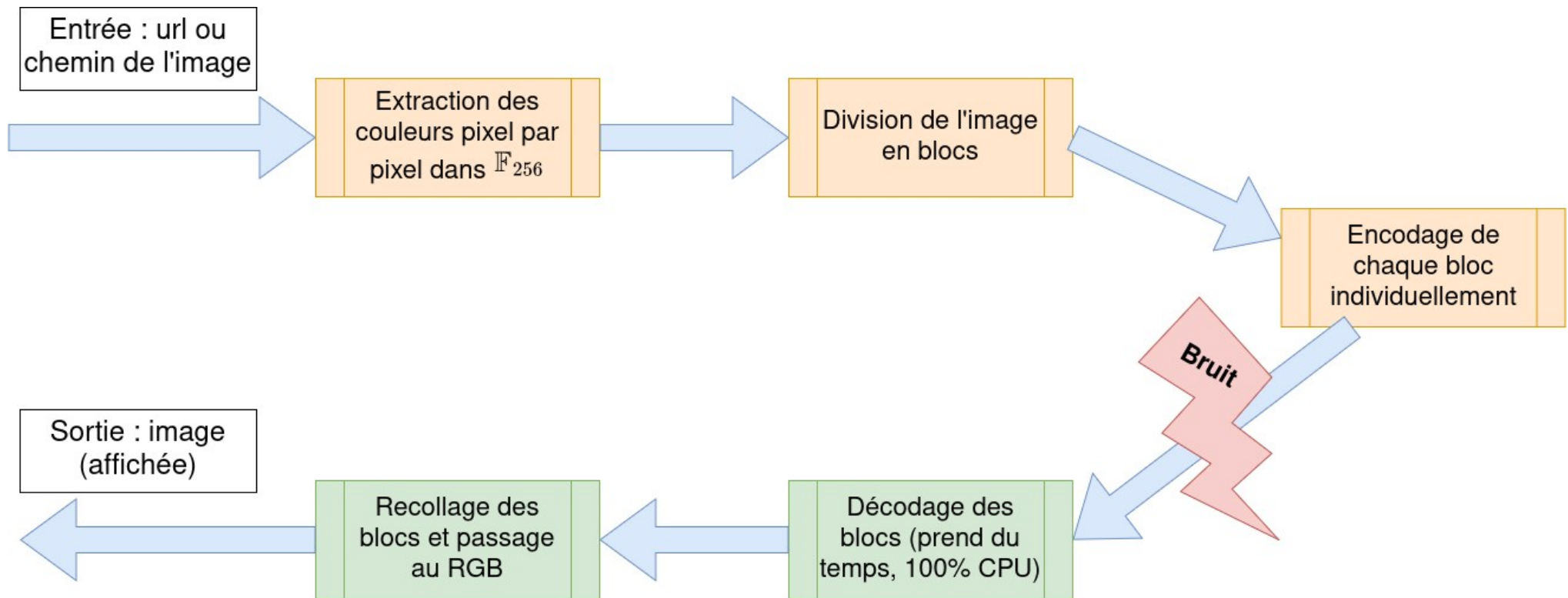
$$\begin{pmatrix} \alpha^{i_1} & \alpha^{i_2} & \dots & \alpha^{i_{n_e}} \\ (\alpha^{i_1})^2 & (\alpha^{i_2})^2 & \dots & (\alpha^{i_{n_e}})^2 \\ \vdots & \vdots & \ddots & \vdots \\ (\alpha^{i_1})^{n_e} & (\alpha^{i_2})^{n_e} & \dots & (\alpha^{i_{n_e}})^{n_e} \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_{n_e} \end{pmatrix} = - \begin{pmatrix} S_1 \\ S_2 \\ \vdots \\ S_{n_e} \end{pmatrix}$$

$$E = D - C = \sum_{r=1}^{n_e} y_r X^{i_r}$$

$A$  est dans les  $k$  premiers coefficients de  $C$ .



### 3) Résultats



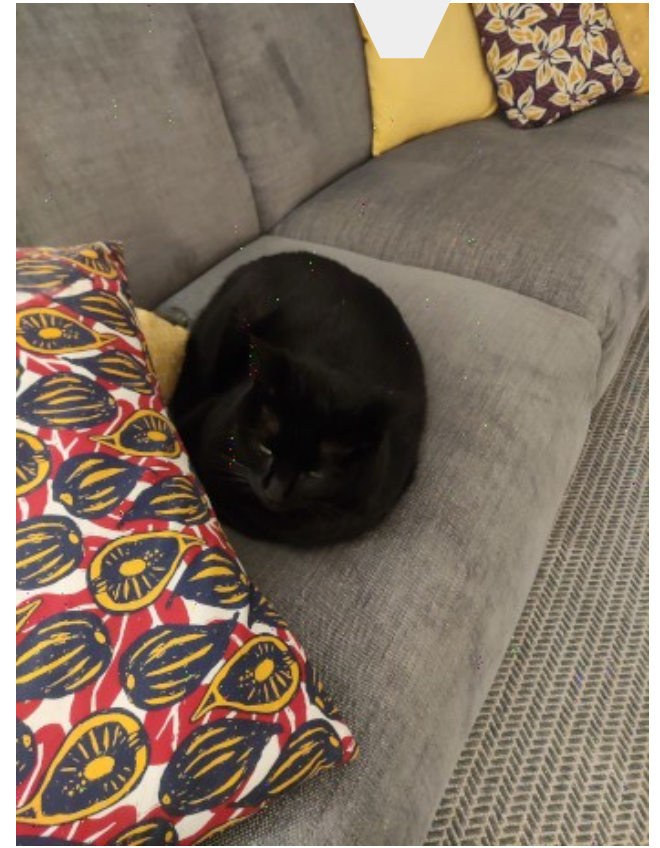
### 3) Résultats



originale



bruit à 6 %

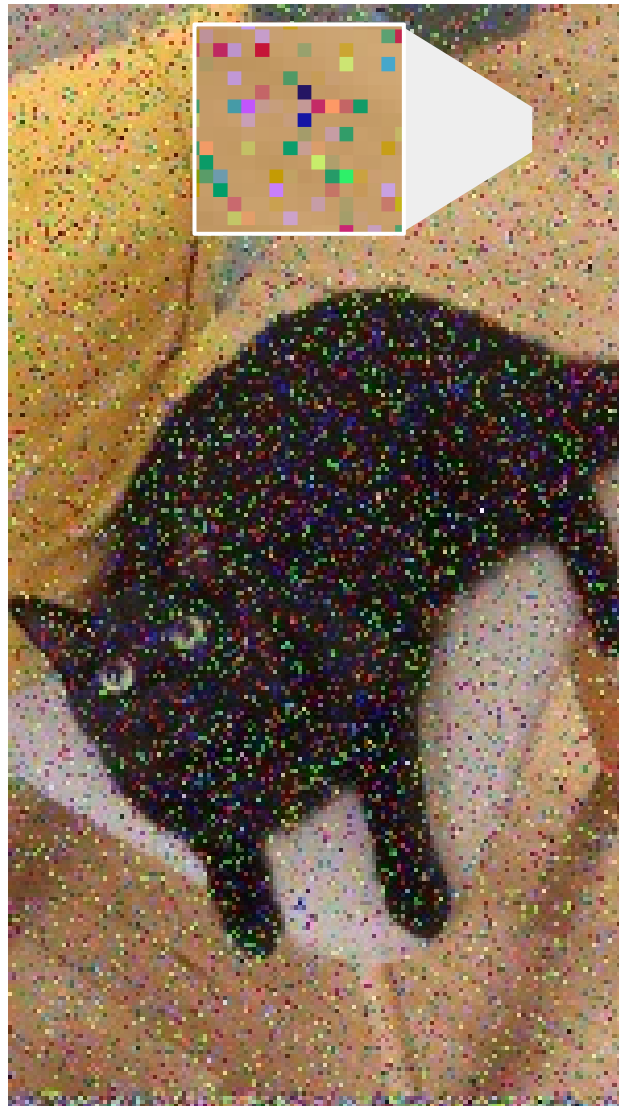


blocs de 12x18,  $t = 19(/19)$   
 $e = 0,045 \%$

### 3) Résultats



originale



bruit à 15 %



bruit à 25 %

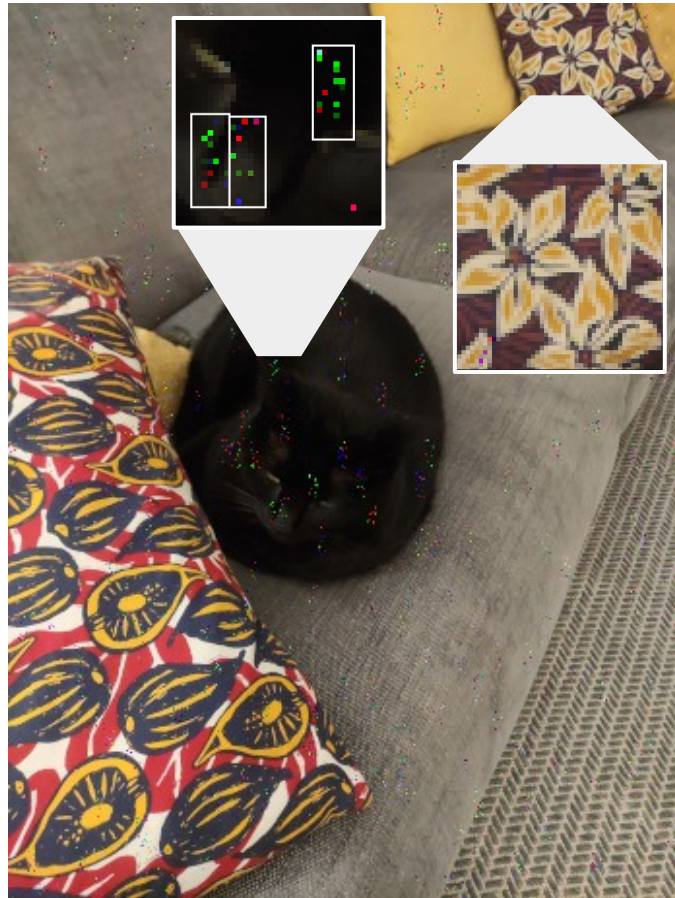


### 3) Résultats

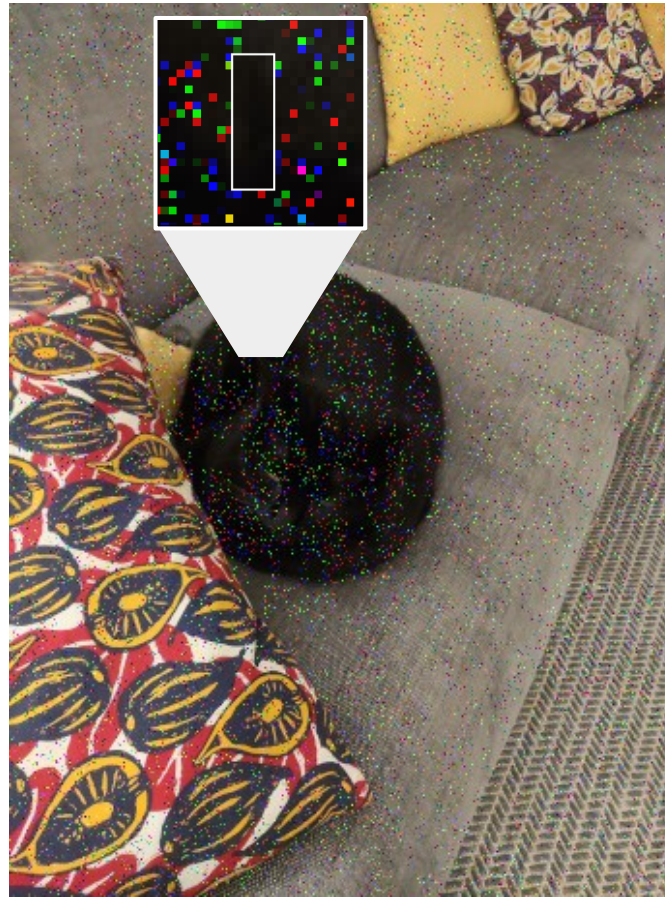
Temps de décodage :

14 s

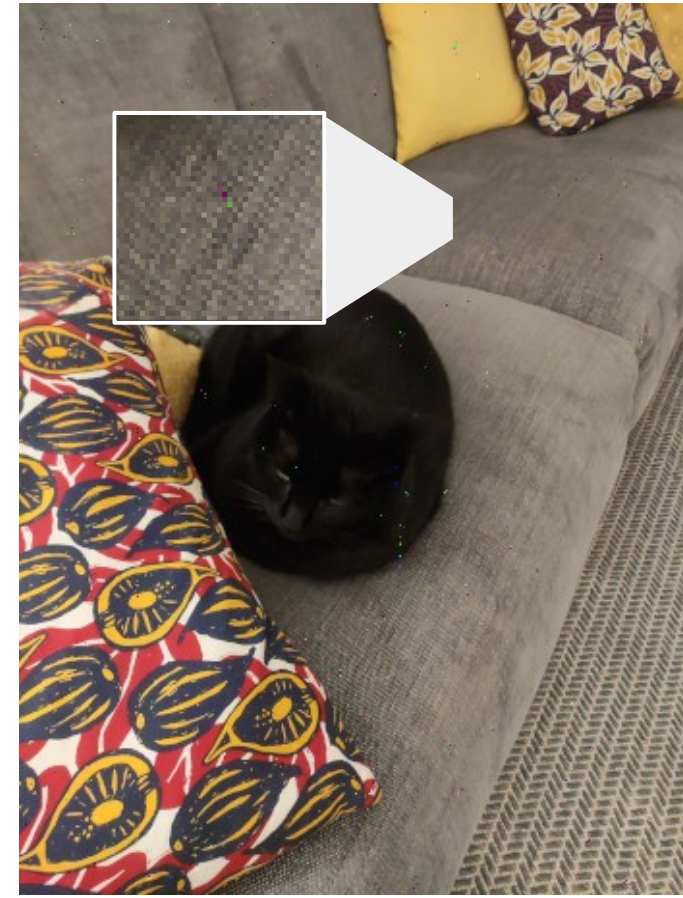
695 s



bruit 15 %, blocs de 15x15,  
 $t = 15$  (/15),  $e = 2,16$  %

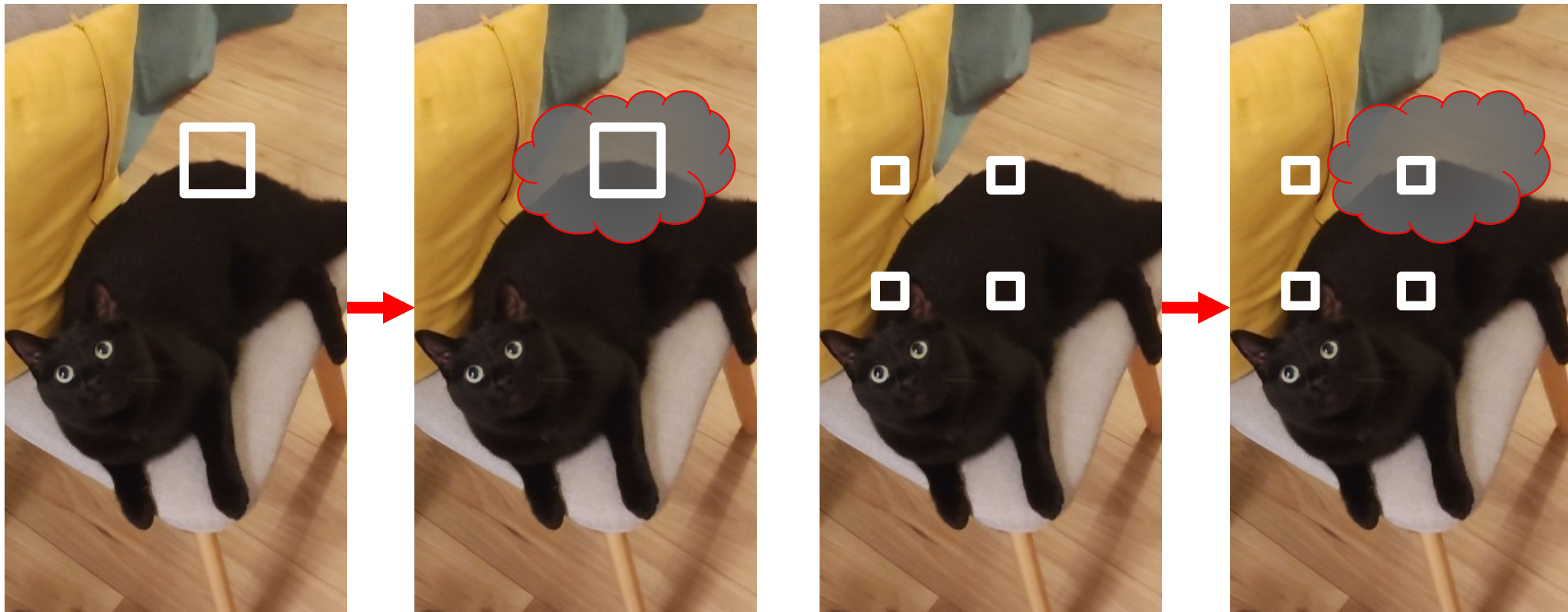


bruit 25 %, blocs de 15x15,  
 $t = 15$  (/15),  $e = 17,4$  %

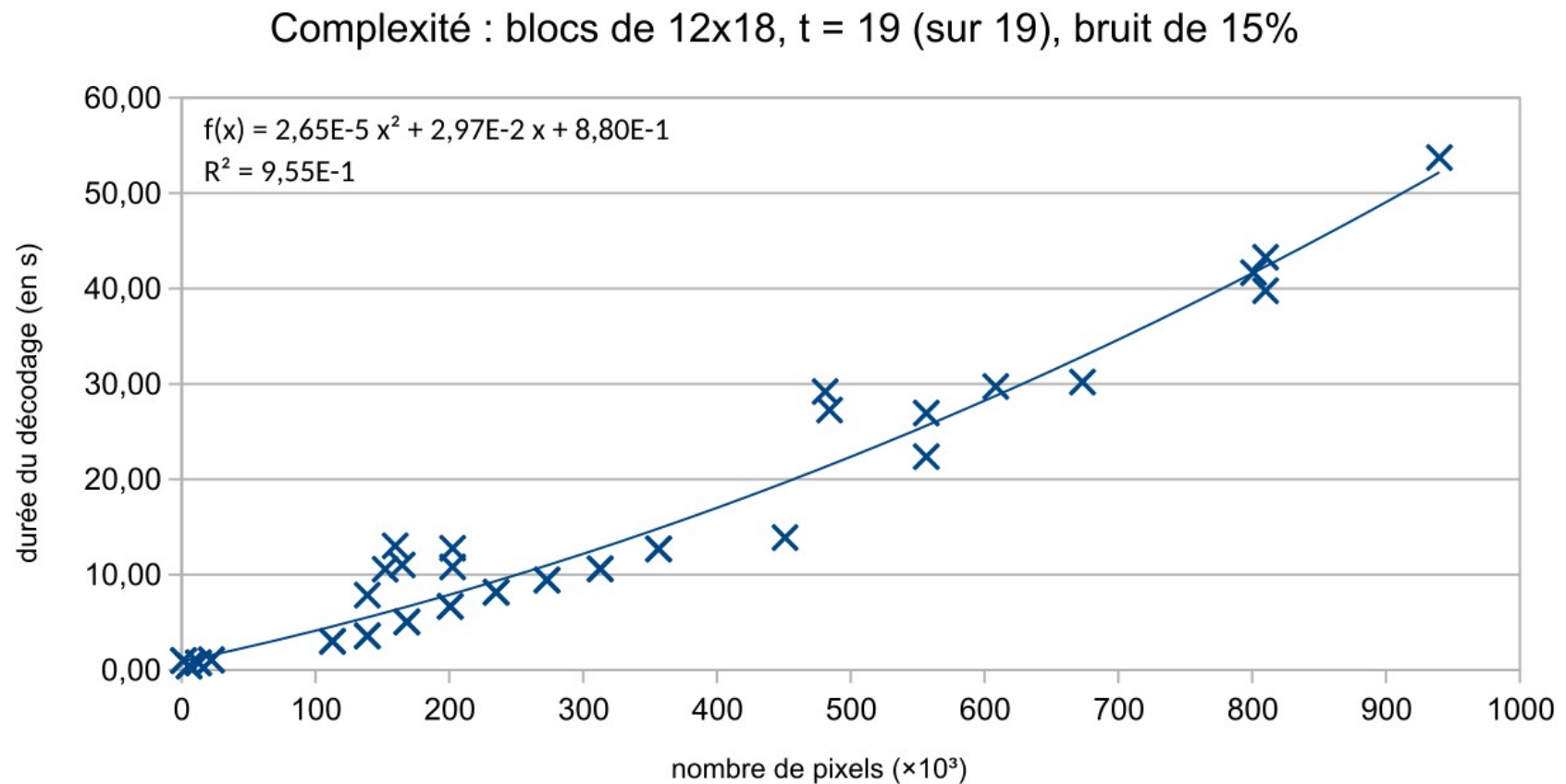


bruit 25 %, blocs de 10x10,  
 $t = 60$  (/70),  $e = 0,42$  %

### 3) Résultats : précisions



## 4) Complexités et paramètres

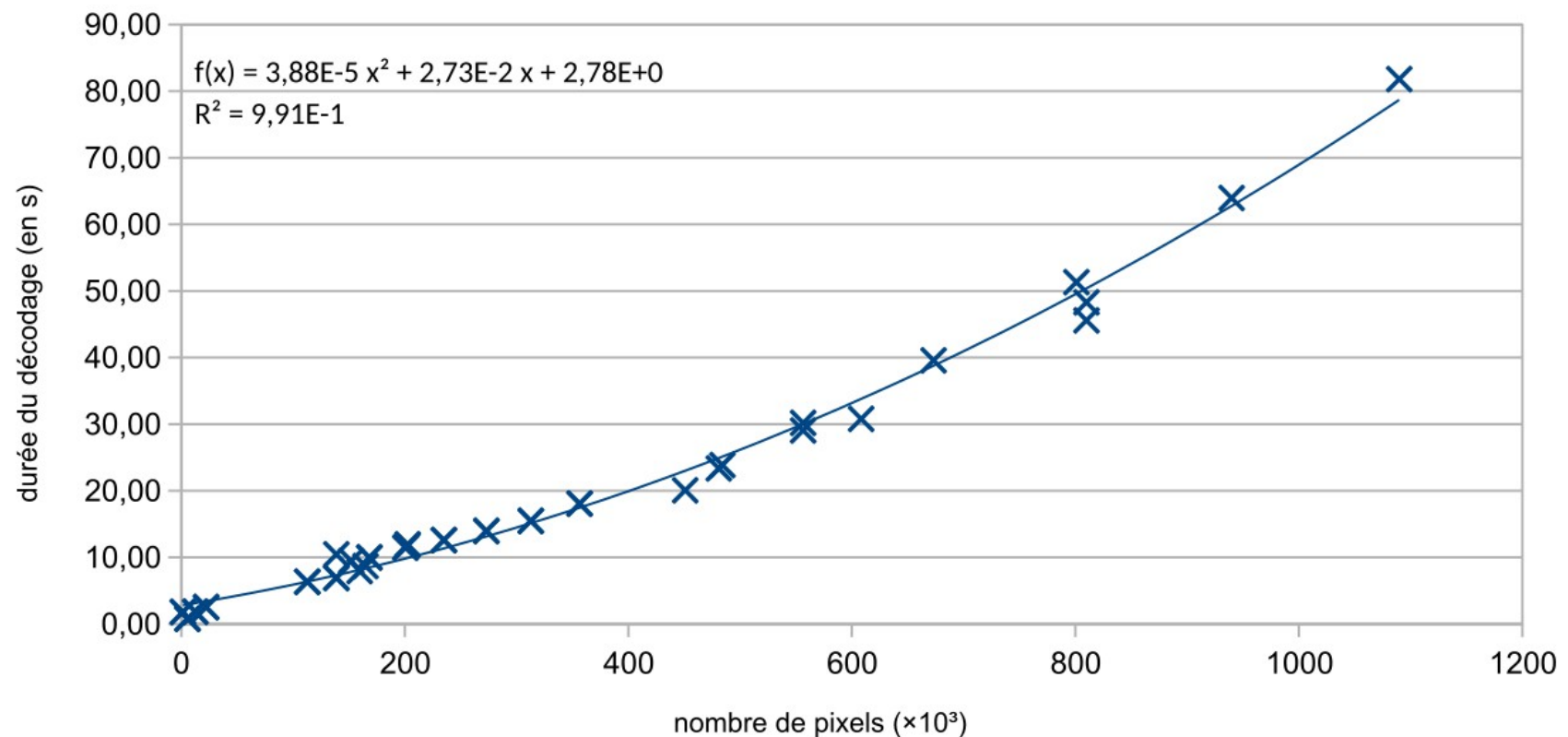


Complexité en :  $O(x^2)$  (x = nombre de pixels)



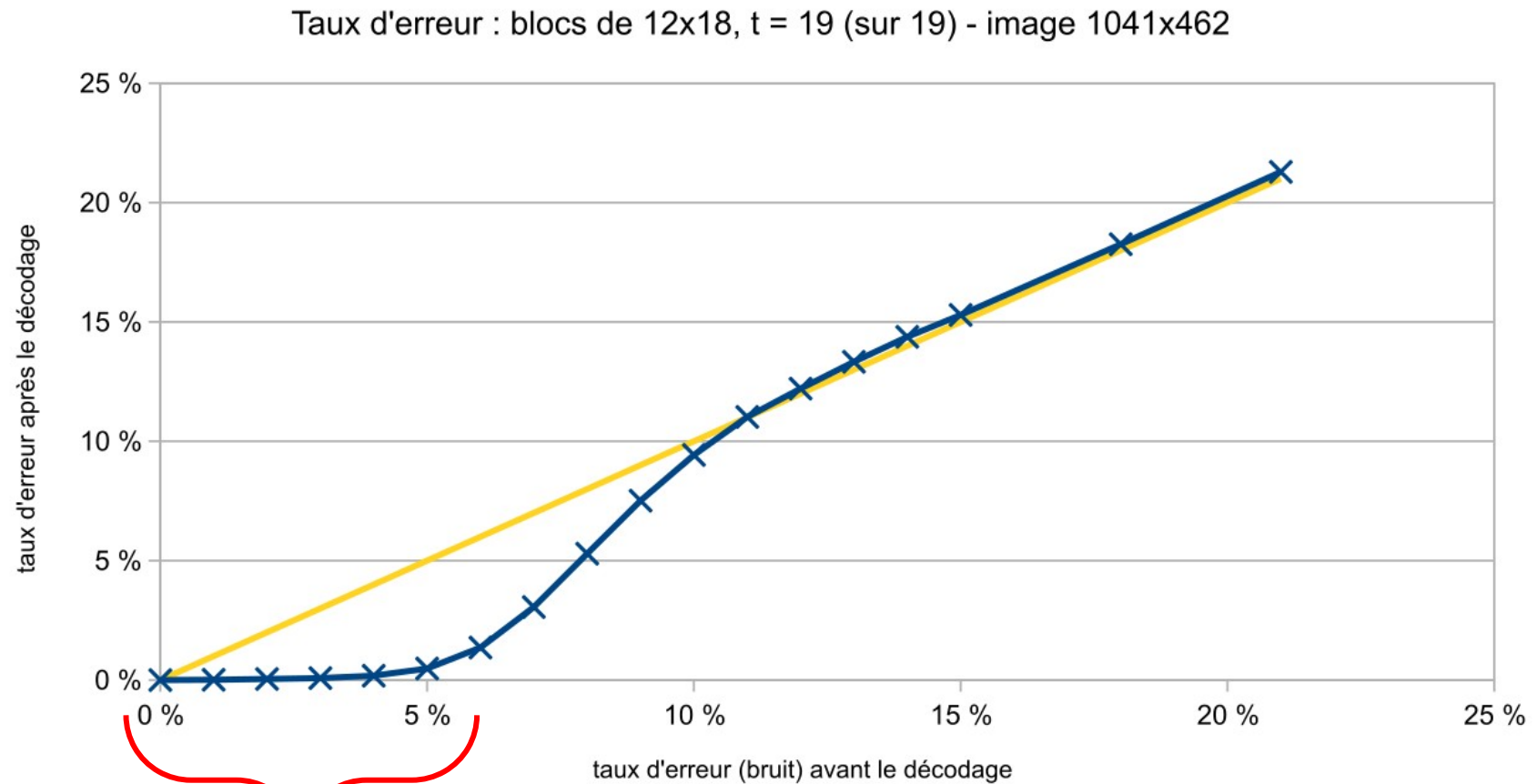
## 4) Complexités et paramètres

Complexité : blocs de 12x18, t = 19 (sur 19), bruit de 4%



Complexité en :  $O(x^2)$  (x = nombre de pixels)

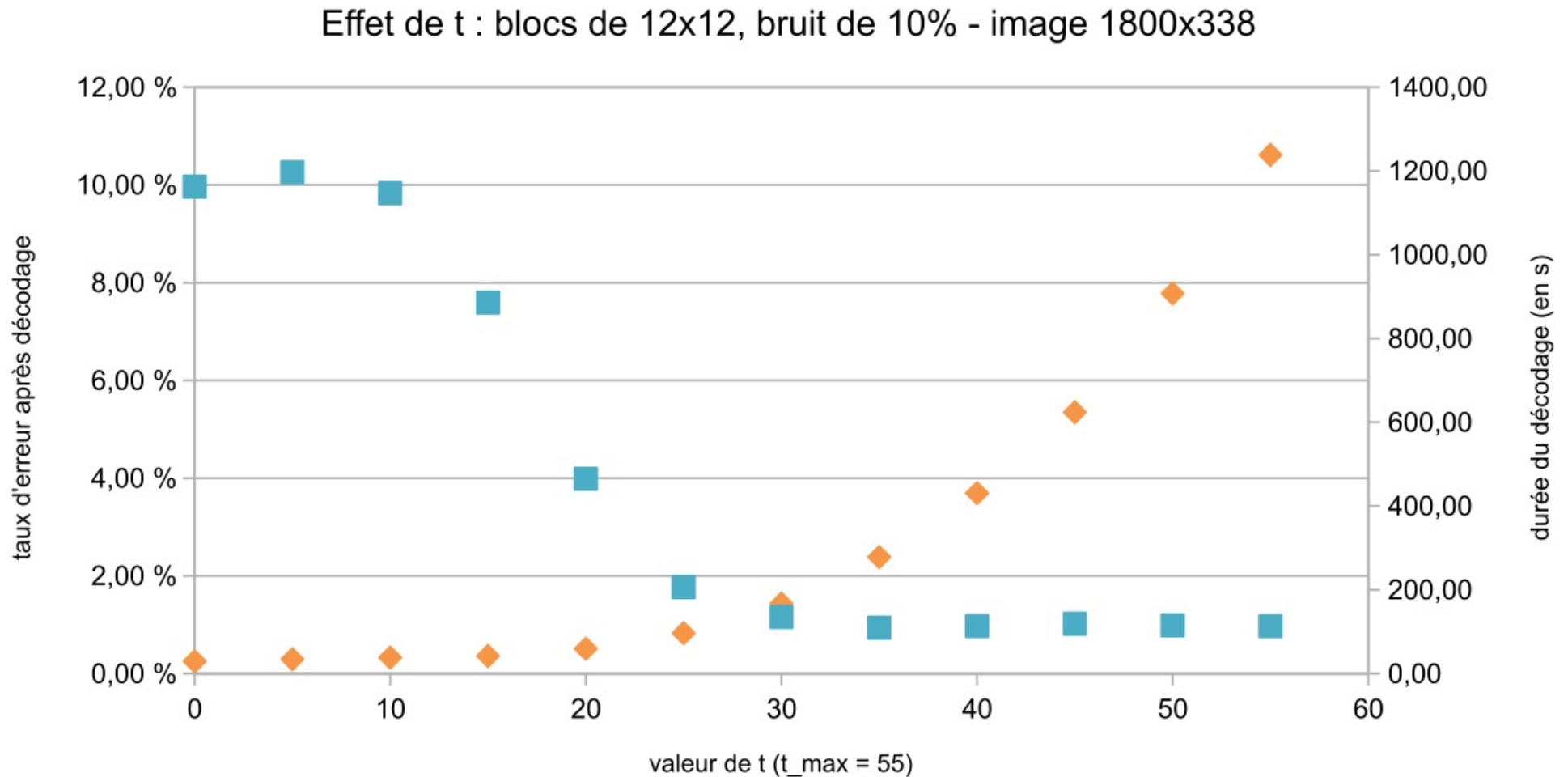
## 4) Complexités et paramètres



Zone optimale qui dépend de  $x$ ,  $y$  et  $t$

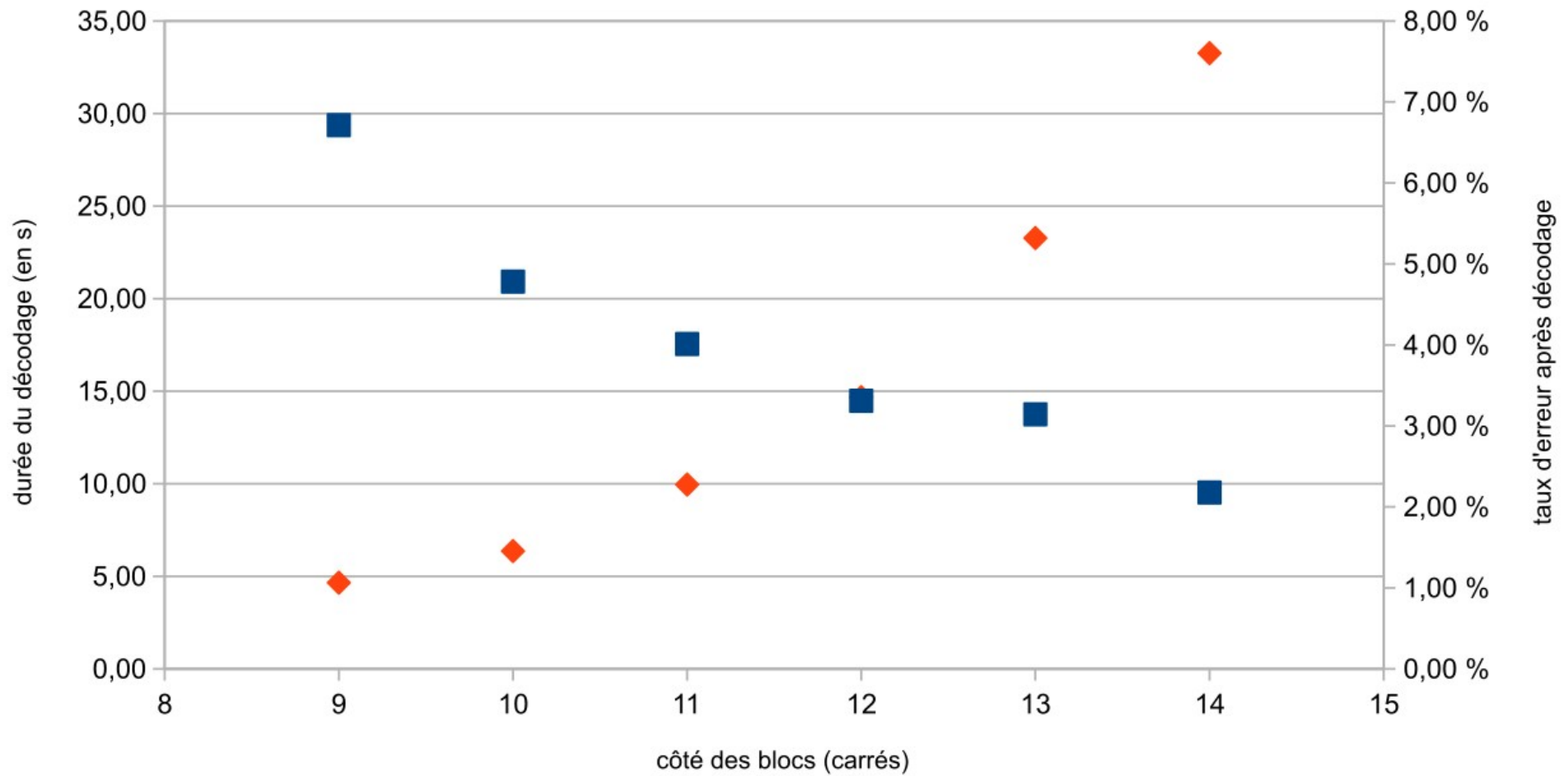


## 4) Complexités et paramètres



## 4) Complexités et paramètres

Effet de la taille des blocs :  $t = 20$  fixé - image 1050x157





**Conclusion  
&  
perspectives**

**Il y aura toujours des pixels erronés si le bruit est trop grand.**

**Cependant, le bruit atteint rarement 10 % mais est de l'ordre de 0,001 % =  $10^{-5}$  pour des communications sans fils standards\*.**

**Il y a eu de nouveaux codes correcteurs  
depuis Reed-Solomon :**

- les Low-Density Parity Check (LDPC)**
- les turbo codes**

**La physique quantique est propice au développement de nouveaux codes correcteurs.**



**Merci de votre attention**

# Annexe



# Le corps $\mathbb{F}_{256}$

$$X^8 + X^7 + X^2 + X + 1 = 110000111$$

(corps de caractéristique 2)

$$\begin{aligned}\overline{X^5 + 1} \times \overline{X^3 + 1} &= \overline{X^8 + X^5 + X^3 + 1} \\ &= \overline{-X^7 - X^2 - X - 1 + X^5 + X^3 + 1} = \overline{X^7 + X^5 + X^3 + X^2 + X} \\ &= \overline{X^{87}} \times \overline{X^{205}} = \overline{X^{292}} = \overline{X^{37}} = \overline{X^7 + X^5 + X^3 + X^2 + X}\end{aligned}$$

$\mathbb{F}_{256}^*$  est un groupe cyclique. On note  $\alpha = \overline{X}$  qui engendre  $\mathbb{F}_{256}^*$



# Gorenstein-Zierler : syndromes

Transmission et décodage : on reçoit  $D \in \mathbb{F}_{256}[X]$ ,  $\deg(D) \leq 255$

Syndromes :  $\forall j \in \llbracket 1, 2t \rrbracket, S_j = D(\alpha^j)$

⚠ Si  $\forall j \in \llbracket 1, 2t \rrbracket, S_j = 0$ ,

$$D = R \times \prod_{j=1}^{2t} (X - \alpha^j) = RG \quad \text{et} \quad D - C = G(R - Q) \equiv 0 [G]$$

*On ne peut pas corriger dans ce cas.*





```

161 #####
162 # ENCODAGE INDIVIDUEL #
163 #####
164
165 def encodage_v2(message, k, t, results, i):
166     P = [F256(0) for j in range(2 * t + 1)] # "i" is not accessed
167     P[-1] = F256(1)
168     controle = div_euclid(prod(message, P), generateur(t))[1]
169     transmis = somme(prod(message, P), controle)
170     transmis += [F256(0)] * (k + 2*t - len(transmis)) # On fait en sorte qu'il est exactement k+2t coefficients
171     results[i] = transmis
172
173 def evaluation(P, a):
174     s = F256(0)
175     for i in range(len(P)):
176         s += P[i] * (a**i)
177     return s
178
179 #####
180 # DECODAGE INDIVIDUEL #
181 #####
182
183 def identite(n):
184     mat = np.full((n, n), F256(0), dtype=F256)
185     for i in range(n):
186         mat[i, i] = F256(1)
187     return mat
188
189 def algo_du_pivot(mat):
190     cop = mat.copy()
191     n = cop.shape[0]
192     n_pivots_prec = 0
193     inv = identite(n)
194     if n != cop.shape[1]:
195         raise Exception("Elle est pas carrée !")
196     for col in range(n):
197         pivot = None
198         # À chaque fois qu'on fait un pivot, on regarde plus que les lignes au dessus
199         for ligne in range(n_pivots_prec, n):
200             pivot = cop[ligne, col]
201             if pivot != F256(0):
202                 # On passe à la suite
203                 break
204         if pivot == F256(0):
205             # Dans ce cas, la matrice n'est pas inversible
206             return False
207         # On échange les lignes "ligne" et "n_pivots_prec"
208         # Il faut mettre des doubles crochets bien échanger les lignes
209         cop[[ligne], cop[[n_pivots_prec]]] = cop[[n_pivots_prec], cop[[ligne]]] #type:ignore
210         inv[[ligne], inv[[n_pivots_prec]]] = inv[[n_pivots_prec], inv[[ligne]]] #type:ignore
211         # Opération Li <- Li - coeff/pivot Lj
212         # et Lj <- coeff/pivot Lj
213         for i in range(n):
214             if cop[i, col] != F256(0):
215                 if i == n_pivots_prec:
216                     cop[i] /= pivot
217                     inv[i] /= pivot
218                     pivot = F256(1)
219                 else:
220                     coeff = cop[i, col] / pivot
221                     cop[i] -= np.multiply(coeff, cop[n_pivots_prec])
222                     inv[i] -= np.multiply(coeff, inv[n_pivots_prec])
223         n_pivots_prec += 1
224     return inv
225
226 def syndromes(recu, t):
227     return [evaluation(recu, table_exp[i]) for i in range(2 * t)]
228
229 def det_poly_correcteur(t, synds):
230     for nu in range(t, 0, -1):
231         mat = np.full((nu, nu), F256(0), dtype=F256)
232         for ligne in range(nu):
233             for col in range(nu):
234                 mat[ligne, col] = synds[nu + ligne - col - 1]
235         inv = algo_du_pivot(mat)
236         if inv is not False: # inv = False si la matrice n'est pas inversible
237             mat_col = np.array(synds[nu:2 * nu]).reshape((nu, 1))
238             lambdas = (inv @ mat_col).reshape((1, nu)).tolist()

```

```

239         return [F256(1)] + lambdas[0]
240     raise Exception("Ya jamais de solution, c'est pas normal")
241
242 def det_ir_xr(poly_lambda):
243     ir, xr = [], []
244     for puissance in range(255):
245         if evaluation(poly_lambda, table_exp[puissance]) == F256(0):
246             ir.append((-puissance) % 255)
247             xr.append(table_exp[(-puissance) % 255])
248     return ir, xr
249
250 def det_E(ir, xr, synds, k, t):
251     nu = len(xr)
252     mat_xr = np.full((nu, nu), F256(0), dtype=F256)
253     for ligne in range(nu):
254         for col in range(nu):
255             mat_xr[ligne, col] = xr[col] ** ligne
256     mat_col = np.array(synds[:nu]).reshape((nu, 1))
257     inv_xr = algo_du_pivot(mat_xr)
258     yr = (inv_xr @ mat_col).reshape((1, nu)).tolist()[0]
259     E = []
260     for i in range(k + 2 * t):
261         if i in ir:
262             E.append(yr.pop())
263         else:
264             E.append(F256(0))
265     return E
266
267 def decodage_v2(recu, k, t, results, i):
268     synds = syndromes(recu, t)
269     correct = True
270     for s in synds:
271         if s != F256(0):
272             correct = False
273             break
274     if correct:
275         results[i] = recu[-k:]
276     else:
277         poly_correcteur = det_poly_correcteur(t, synds)
278         ir, xr = det_ir_xr(poly_correcteur)
279         E = det_E(ir, xr, synds, k, t)
280         s = somme(recu, E)[-k:]
281         s += [F256(0)] * (k - len(s))
282         results[i] = s
283
284 #####
285 # MULTIPROCESS #
286 #####
287
288 # Execute sur chaque bloc une fonction f qui respecte le bon format
289 # Utilise pour l'encodage et le décodage
290 def multiprocess(blocs, f, t, n_process, silent):
291     n_blocs = len(blocs)
292     blocs_traites = [None] * n_blocs
293     # Gestion des processus en parallèle
294     process = [None] * n_process
295     manager = Manager()
296     results = manager.list([None] * n_process)
297     coords_list = [None] * n_process
298
299     for g in range(n_blocs // n_process):
300         if g == 0:
301             debut = time.time()
302             # Lancement de chaque processus
303             for i in range(n_process):
304                 empla = n_process * g + i
305                 coords_list[i] = blocs[empla][1]
306                 k_i = coords_list[i][2] * coords_list[i][3]
307                 process[i] = Process(target=f, args=(blocs[empla][0].tolist(), k_i, t, results, i))
308                 process[i].start()
309             for i in range(n_process):
310                 process[i].join()
311                 empla = n_process * g + i
312                 # Stockage des résultats
313                 blocs_traites[empla] = (np.array(results[i]), coords_list[i]) #type:ignore
314         if g == 0:
315             fin = time.time()
316             if not silent:
317                 print("Temps estimé :", int((fin - debut) * (n_blocs // n_process)), "secondes.") #type:ignore

```



```

321 # S'il reste quelques processus on les fait (n_process peut ne pas diviser n_blocs)
322 m = n_blocs % n_process
323 for g in range(m):
324     for i in range(m):
325         empla = n_blocs - i - 1
326         coords_list[i] = blocs[empla][1]
327         k_i = coords_list[i][2] * coords_list[i][3]
328         process[i] = Process(target=f, args=(blocs[empla][0].tolist(), k_i, t, results, i))
329         process[i].start()
330     for i in range(m):
331         process[i].join()
332     empla = n_blocs - i - 1
333     blocs_traites[empla] = (np.array(results[i]), coords_list[i]) #type: ignore
334
335     return blocs_traites
336
337 #####
338 # ENCODAGE IMAGE #
339 #####
340
341 # Récupère une image d'une URL et nous renvoie une matrice d'éléments de F256 la représentant.
342 def extract_img(url):
343     # img = Image.open(urlopen(url))
344     img = Image.open(url)
345     largeur, hauteur = img.size
346     pixels = list(img.get_flattened_data())
347     img.close()
348     # RGB fait tripler la largeur de l'image
349     Fpixels = np.full((hauteur, largeur * 3), F256(0), dtype=F256)
350     for i in range(len(pixels)):
351         x = i % largeur
352         y = i // largeur
353         for c in range(3):
354             Fpixels[y, 3 * x + c] = F256(pixels[i][c]) #type: ignore
355     return Fpixels
356
357 # NOTE
358 # Ce n'est pas un problème que la taille des blocs soit différente
359 # alors qu'ils ont le même t.
360
361 # Prend une matrice de F256 et la divise en blocs.
362 # Les blocs peuvent ne pas quadriller parfaitement.
363 # Les blocs sont des blocs de composantes de couleur et non de pixels
364 def diviser_blocs(img_src, blocs_x, blocs_y):
365     img_y, img_x = img_src.shape
366     x, y = 0, 0
367     blocs = []
368     while True:
369         # Les coordonnées et dimensions réelles du bloc (tenant compte des bords)
370         coords = (
371             x, y,
372             blocs_x if x + blocs_x <= img_x else img_x - x,
373             blocs_y if y + blocs_y <= img_y else img_y - y
374         )
375         bloc_xy = img_src[y:y+coords[3], x:x+coords[2]]
376         blocs.append((bloc_xy.flatten(), coords))
377         # Coin en bas à droite
378         if x + blocs_x >= img_x and y + blocs_y >= img_y:
379             break
380         # Côté de droite
381         if x + blocs_x < img_x:
382             x += blocs_x
383         else:
384             x, y = 0, y + blocs_y
385     return blocs
386
387 def encoder_blocs(blocs, t, n_process, silent):
388     return multiprocessing(blocs, encodage_v2, t, n_process, silent)
389
390 #####
391 # DÉCODAGE IMAGE #
392 #####
393
394 # Changer un certain % de valeurs de couleurs aléatoirement
395 def bruit(blocs_encodes, pourcent):
396     # On calcule dans un premier temps la taille réelle du message transmis
397     img_trans_size = 0

```

```

398     chg = 0
399     array_bruite = []
400     for bloc, coords in blocs_encodes:
401         img_trans_size += len(bloc)
402         array_bruite.append((bloc.copy(), coords))
403     # On s'assure de modifier le bon pourcentage en:
404     # ne modifiant qu'une seule valeur de rgb
405     # en vérifiant qu'on n'a pas déjà modifié le pixel
406     while chg < img_trans_size * pourcent:
407         b_rand = random.randint(0, len(blocs_encodes)-1)
408         val_rand = random.randint(0, len(blocs_encodes[b_rand][0])-1)
409         if array_bruite[b_rand][0][val_rand] == blocs_encodes[b_rand][0][val_rand]:
410             array_bruite[b_rand][0][val_rand] = F256(random.randint(0, 255))
411             chg += 1
412     return array_bruite
413
414 def decoder_blocs(blocs, t, n_process, silent):
415     return multiprocessing(blocs, decodage_v2, t, n_process, silent)
416
417 # On récupère une image en bloc et on la reconstruit selon les bonnes dimensions toujours en F256
418 def recreer_img(blocs, img_x, img_y):
419     img = np.full((img_y, img_x), 0, dtype=F256)
420     for bloc, coords in blocs:
421         img[coords[1]:coords[1] + coords[3], coords[0]:coords[0] + coords[2]] = bloc.reshape((coords[3], coords[2]))
422     return img
423
424 # On récupère une image en F256 et on fait une image PIL
425 def PIL_img(img):
426     img_y, img_x = img.shape
427     if img_x % 3 != 0:
428         raise Exception("R, G et B ne sont pas à la suite")
429     rgb_img = np.full((img_y, img_x // 3, 3), 0)
430     for y in range(img_y):
431         for x in range(img_x // 3):
432             for c in range(3):
433                 rgb_img[y, x, c] = img[y, 3 * x + c].poly
434     PIL_img = Image.fromarray(rgb_img.astype("uint8"), "RGB")
435     return PIL_img
436
437 # On récupère une image résultat en F256 et l'image de base (en F256)
438 # et on renvoie le % d'erreur en couleur et en pixel
439 def erreur(final, initial):
440     compte_val = 0
441     compte_pixel = 0
442     if (len(final), len(final[0])) != (len(initial), len(initial[0])):
443         raise Exception("Les tailles ne correspondent pas ! (force à toi)")
444     for y in range(len(final)):
445         pixel = 0
446         erreur_pixel = False
447         for x in range(len(final[0])):
448             if final[y][x] != initial[y][x]:
449                 compte_val += 1
450                 if not erreur_pixel:
451                     erreur_pixel = True
452                     compte_pixel += 1
453         pixel += 1
454         if pixel == 3:
455             pixel = 0
456             erreur_pixel = False
457     size = len(final) * len(final[0])
458     return (compte_val / size, compte_pixel / size)
459
460 #####
461 # ANCIENNE VERSION #
462 #####
463
464 def encodage_v1(u):
465     k = u.shape[0]
466     a = np.zeros(255, dtype=F256)
467     for i in range(255):
468         c = F256(0)
469         for j in range(k):
470             c += u[j] * table_exp[(i * j) % 255]
471         a[i] = c
472     return a
473
474 def combi_random(jusqua, k, iter_max):
475     for i in range(iter_max):
476         yield random.sample(range(jusqua), k)

```

```

480 # Pour la suite, il est nécessaire que decodage ne renvoie pas de données
481 # mais modifie une liste en argument à l'emplacement prévu
482 def decodage_v1(w, k, lim_pointe, lim_max, liste, emplacement):
483     udict = {}
484     # Pour chaque façon de prendre k équations parmi les 255
485     for kcombi in combi_random(255, k, lim_max):
486         w_k = np.array([w[x] for x in kcombi])
487         matrice_systeme = np.zeros((k, k), dtype=F256)
488         for i in range(len(kcombi)):
489             for puissance in range(k):
490                 matrice_systeme[i, puissance] = table_exp[(kcombi[i] * puissance) % 255]
491         u = algo_du_pivot(matrice_systeme) @ w_k
492         # On convertit u en un object "hashable"
493         hash_u = tuple([int(i) for i in u])
494         if hash_u not in udict:
495             udict[hash_u] = 0
496         udict[hash_u] += 1
497         # Dès que l'on a plus de "limite" équations menant au même vecteur on s'arrête
498         max_compte = 0
499         max_u = None
500         for u in udict:
501             if udict[u] > max_compte:
502                 max_compte = udict[u]
503                 max_u = u
504         if max_compte >= lim_pointe:
505             liste[emplacement] = np.array(max_u)
506             return
507     raise Exception("pas trouvé")
508

```